

RAPPORT DE PROJET DEVOPS

Deploying an Open-Source Large Language Model (LLM)

Ollama • Open WebUI • Minikube

Étudiante	Ndeye Absa DER
Environnement	Windows, Docker Desktop, Minikube
Namespace Kubernetes	llm-app
Modèles déployés	llama3.2:3b, mistral:7b, gemma3:4b

Rapport final



1. Introduction

L'objectif de ce projet est de déployer une plateforme locale d'intelligence artificielle basée sur des modèles de langage open source. L'application repose sur deux composants complémentaires : Ollama, chargé d'héberger et d'exécuter les modèles, et Open WebUI, qui fournit une interface web permettant de sélectionner un modèle, d'envoyer un prompt et de consulter la réponse générée.

Après une première prise en main avec Docker Compose, l'architecture a été recréée avec Kubernetes sur un cluster Minikube. Le déploiement applique les bonnes pratiques demandées : isolation dans un namespace, configuration externalisée, stockage persistant, contrôle des ressources et sondes de santé.

2. Architecture et ressources Kubernetes

2.1 Namespace et ConfigMap

Le namespace `llm-app` regroupe toutes les ressources du projet. Cette isolation simplifie les commandes d'administration, la lecture des événements et le nettoyage de l'environnement. Une ConfigMap centralise les paramètres non sensibles, notamment l'adresse utilisée par Open WebUI pour joindre Ollama. La configuration n'est donc pas intégrée directement dans l'image du conteneur.

2.2 Deployments et Services

Deux Deployments maintiennent les pods applicatifs. Le Deployment Ollama utilise l'image `ollama/ollama:latest` et expose l'API sur le port 11434. Le Deployment Open WebUI utilise l'image `ghcr.io/open-webui/open-webui:main` et expose l'interface sur le port 8080. Les Deployments apportent un fonctionnement déclaratif et recréent automatiquement un pod en cas de suppression ou de panne.

Deux Services assurent un accès stable aux pods : `ollama-service` est de type `ClusterIP` et reste accessible uniquement dans le cluster, tandis que `webui-service` est de type `NodePort`. Le service web expose le port 8080 sur le port 30080 du nœud Minikube.

Utilisateur → webui-service:30080 → Open WebUI:8080 → ollama-service:11434 → Ollama → Modèles LLM

2.3 Stockage persistant

Deux PersistentVolumeClaims ont été créés. `ollama-pvc` conserve les modèles téléchargés dans le répertoire de données d'Ollama. `webui-pvc` conserve les comptes, les conversations et les paramètres d'Open WebUI. Sans ces volumes, une recréation des pods entraînerait la perte des modèles et des informations applicatives. Les PVC garantissent donc la continuité des données au-delà du cycle de vie des conteneurs.

3. Communication entre Open WebUI et Ollama

Open WebUI ne contacte pas directement l'adresse IP du pod Ollama, car cette adresse peut changer.

L'interface utilise le nom DNS interne `ollama-service` et le port 11434. Kubernetes associe ce Service au pod portant le label d'Ollama. Lorsqu'un utilisateur soumet un prompt, Open WebUI transmet la demande à l'API Ollama, Ollama exécute le modèle sélectionné, puis renvoie la réponse à l'interface.

4. Fiabilité et gestion des ressources

4.1 Requests et limits

Chaque conteneur possède des requests et des limits CPU et mémoire. Les requests indiquent au planificateur Kubernetes les ressources minimales nécessaires. Les limits empêchent un conteneur de consommer sans contrôle les ressources du nœud. Cette configuration est particulièrement importante pour Ollama, car l'inférence de modèles de plusieurs gigaoctets peut solliciter fortement la mémoire et le processeur.

4.2 Sondes de santé

Trois types de sondes ont été configurés :

- La startup probe laisse le temps à l'application de terminer son initialisation avant l'application des autres contrôles.
- La readiness probe indique si le pod est prêt à recevoir des requêtes. Un pod non prêt est retiré temporairement des points de terminaison du Service.
- La liveness probe détecte un conteneur bloqué ou non fonctionnel et permet à Kubernetes de le redémarrer automatiquement.

L'utilisation combinée de ces sondes limite les interruptions et évite d'envoyer du trafic vers une application qui n'est pas encore disponible.

5. Validation du déploiement

Le déploiement a été validé sur Minikube. Les pods Ollama et Open WebUI ont atteint l'état Running avec un conteneur prêt sur un conteneur. Les services ont été créés avec les paramètres suivants :

Service	Type	Port du service	Exposition
ollama-service	ClusterIP	11434/TCP	Interne au cluster
webui-service	NodePort	8080/TCP	NodePort 30080

Trois modèles ont également été téléchargés dans Ollama et sont disponibles dans le stockage persistant : llama3.2:3b (2,0 Go), mistral:7b (4,4 Go) et gemma3:4b (3,3 Go). Leur présence a été vérifiée avec la commande `ollama list` exécutée dans le Deployment Ollama.

6. Docker Compose et Kubernetes

Docker Compose est adapté au lancement rapide d'une application multi-conteneurs sur une seule machine. Sa configuration est simple et convient bien au développement local. Kubernetes propose une orchestration plus complète : objets déclaratifs, découverte de services, stockage persistant, sondes de santé, contrôle des ressources et auto-réparation des pods.

Dans ce projet, Docker Compose facilite la validation locale initiale. Kubernetes apporte ensuite une structure plus robuste et plus proche d'un environnement de production. Le passage de l'un à l'autre nécessite de transformer chaque composant en ressources Kubernetes distinctes et explicitement configurées.

7. Conclusion

Ce projet a permis de déployer avec succès une application d'intelligence artificielle conteneurisée sur Kubernetes. L'architecture obtenue sépare clairement l'interface Open WebUI du moteur Ollama, tout en assurant leur communication par les Services Kubernetes. Les PVC protègent les modèles et les données applicatives, tandis que les requests, limits et sondes de santé améliorent la stabilité du déploiement.

Les résultats obtenus répondent aux objectifs de la partie Kubernetes : ressources YAML déployées, pods opérationnels, services accessibles et trois modèles disponibles. Les captures des pods, des services, de la page d'accueil et des réponses générées complètent les preuves de validation à joindre au dépôt GitHub.